

Documentación Técnica Detallada del Servicio Priority (gRPC)

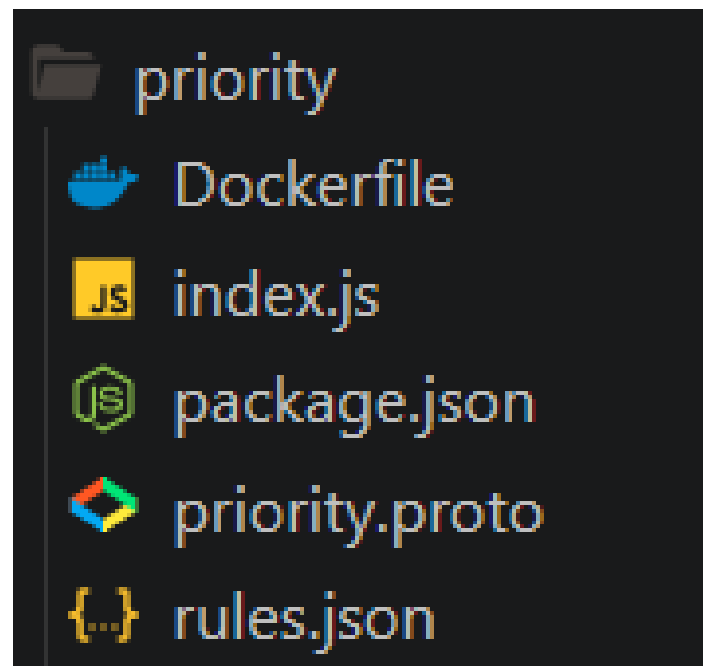
Descripción General

El servicio **Priority** es un microservicio desarrollado en **Node.js** que forma parte del sistema **Campus Universitario Inteligente (C5 Campus)**.

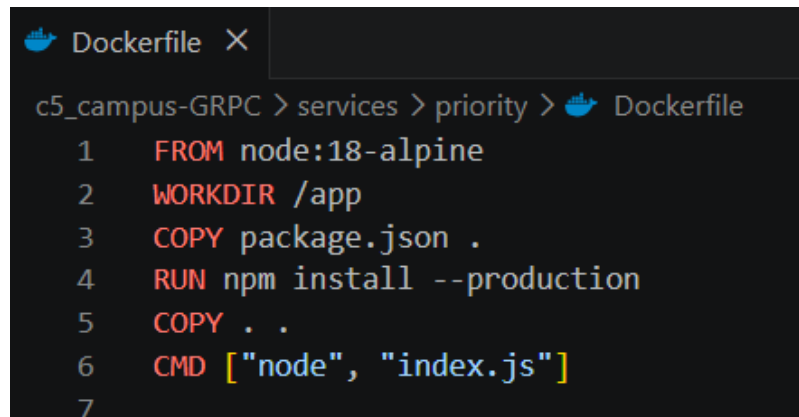
Su función principal es:

- Recibir alertas de emergencia desde otros microservicios.
- Clasificar automáticamente la prioridad de la emergencia.
- Devolver una respuesta rápida mediante gRPC.
- Permitir la administración dinámica de reglas mediante una API REST.
- Utilizar una **Tabla Hash O (1)** para lograr tiempos de clasificación extremadamente rápidos

Estructura de la carpeta de Priority



Dockerfile



```
Dockerfile X
c5_campus-GRPC > services > priority > Dockerfile
1 FROM node:18-alpine
2 WORKDIR /app
3 COPY package.json .
4 RUN npm install --production
5 COPY . .
6 CMD ["node", "index.js"]
7
```

El archivo **Dockerfile** es el encargado de definir el entorno donde se ejecutará el microservicio de prioridad. Su propósito es automatizar la creación de una imagen Docker que contenga todas las dependencias, configuraciones y archivos necesarios para que la aplicación funcione de manera consistente en cualquier servidor o computadora.

La instrucción **FROM node:18-alpine** establece la imagen base sobre la cual se construirá el contenedor. En este caso se utiliza la imagen oficial de Node.js en su versión 18 junto con Alpine Linux, una distribución ligera que reduce significativamente el tamaño final de la imagen y mejora el rendimiento durante el despliegue, gracias a esto, el contenedor consume menos recursos y puede iniciarse más rápidamente.

Posteriormente, la instrucción **WORKDIR /app** define el directorio de trabajo principal dentro del contenedor, todas las operaciones que se ejecuten después de esta línea tendrán como referencia la carpeta /app. Si dicha carpeta no existe, Docker la crea automáticamente durante el proceso de construcción.

La instrucción **COPY package.json .** copia el archivo package.json desde el proyecto local hacia el directorio de trabajo del contenedor, este paso se realiza antes de copiar el resto de los archivos para aprovechar el sistema de caché de Docker, de esta manera, si únicamente se modifica el código fuente pero no las dependencias, Docker puede reutilizar capas previamente construidas y acelerar considerablemente el proceso de creación de la imagen.

A continuación, la instrucción **RUN npm install --production** instala todas las dependencias necesarias para la ejecución del servicio, el parámetro --production indica que únicamente se instalarán los paquetes requeridos en el entorno de producción, excluyendo herramientas de desarrollo o pruebas. Entre las dependencias instaladas se encuentran Express para la API REST, gRPC para la comunicación entre microservicios,

Proto Loader para cargar los archivos .proto y Redis para futuras integraciones con sistemas de mensajería o almacenamiento en caché.

La instrucción **COPY . .** copia todos los archivos restantes del proyecto al contenedor. Esto incluye el archivo principal index.js, el archivo de definición gRPC priority.proto, el archivo de configuración rules.json y cualquier otro recurso necesario para el funcionamiento del servicio.

Finalmente, la instrucción **CMD ["node", "index.js"]** define el comando que será ejecutado automáticamente cuando el contenedor se inicie, este comando pone en marcha la aplicación Node.js, iniciando tanto el servidor gRPC como la API REST, además de cargar las reglas de clasificación y construir la tabla hash utilizada para realizar búsquedas de prioridad en tiempo constante $O(1)$.

Index.js

```
index.js X
c5_campus-GRPC > services > priority > index.js > app.get("/health") callback
1  const grpc      = require("@grpc/grpc-js");
2  const protoLoader = require("@grpc/proto-loader");
3  const Redis      = require("ioredis");
4  const express    = require("express");
5  const fs         = require("fs");
6  const path       = require("path");
7
8  const app = express();
9  app.use(express.json());
10
11 const GRPC_PORT = process.env.GRPC_PORT || 50051;
12 const HTTP_PORT = process.env.HTTP_PORT || 3003;
13 const REDIS_URL = process.env.REDIS_URL || "redis://localhost:6379";
14 const RULES_PATH = path.join(__dirname, "rules.json");
15 const redis = new Redis(REDIS_URL);
16
17 let lookupTable = {};
18 let rulesConfig = {};
19 let rulesVersion = "";
20
21 function buildLookupTable(rules) {
22   const table = {};
23   for (const [priority, config] of Object.entries(rules)) {
24     if (!config.types) continue;
25     for (const type of config.types) {
26       table[type.toLowerCase().trim()] = {
27         priority,
28         label: config.label,
29         reason: config.reason,
30         response_time_seconds: config.response_time_seconds,
31         color: config.color,
32       };
33     }
34   }
35   return table;
36 }
```

```

indexjs X
c5_campus-GRPC > services > priority > indexjs > classifyAlert
37
38 function loadRules() {
39   try {
40     const raw = fs.readFileSync(RULES_PATH, "utf8");
41     const rules = JSON.parse(raw);
42
43     lookupTable = buildLookupTable(rules);
44     rulesConfig = rules;
45     rulesVersion = rules._version || Date.now().toString();
46
47     const total = Object.keys(lookupTable).length;
48     console.log(`[Priority] ✓ Reglas cargadas v${rulesVersion} - ${total} tipos indexados en tabla hash`);
49     console.log(`[Priority] Critico: ${rules.critical?.types?.length || 0} tipos`);
50     console.log(`[Priority] Alto: ${rules.high?.types?.length || 0} tipos`);
51     console.log(`[Priority] Medio: ${rules.medium?.types?.length || 0} tipos`);
52   } catch (err) {
53     console.error(`[Priority] Error cargando rules.json:`, err.message);
54     process.exit(1);
55   }
56 }
57
58 fs.watch(RULES_PATH, (event) => {
59   if (event === "change") {
60     console.log(`[Priority] 🔄 rules.json modificado - recargando en caliente...`);
61     setTimeout(loadRules, 200);
62   }
63 });
64
65 function classifyAlert(emergencyType) {
66   const start = process.hrtime.bigint();
67
68   const key = (emergencyType || "").toLowerCase().trim();
69   const match = lookupTable[key];
70
71   const elapsed = Number(process.hrtime.bigint() - start);
72
73   if (match) {
74     return {
75       priority: match.priority,
76       label: match.label,
77       reason: match.reason,
78       response_time_seconds: match.response_time_seconds,
79       color: match.color,
80       classification_ns: elapsed,

```

```

indexjs X
c5_campus-GRPC > services > priority > indexjs > classifyAlert
65 function classifyAlert(emergencyType) {
81   matched_type: key,
82   };
83 }
84
85 return {
86   priority: rulesConfig.default_priority || "medium",
87   label: "MEDIO",
88   reason: rulesConfig.default_reason,
89   response_time_seconds: 1800,
90   color: "#FFD700",
91   classification_ns: elapsed,
92   matched_type: null,
93   };
94 }
95
96 const packageDef = protoloader.loadSync(
97   path.join(__dirname, "priority.proto"),
98   { keepCase: true, longs: String, enums: String, defaults: true, oneofs: true }
99 );
100 const proto = grpc.loadPackageDefinition(packageDef).priority;
101
102 function ClassifyAlert(call, callback) {
103   const { device_id, emergency_type, zone } = call.request;
104   const result = classifyAlert(emergency_type);
105
106   console.log(
107     `[Priority gRPC] + device=${device_id} | ` +
108     `tipo="${emergency_type}" | zona="${zone}" | ` +
109     `→ ${result.priority.toUpperCase()} (${result.classification_ns}ns)`
110   );
111
112   callback(null, {
113     alert_id: `ALT-${Date.now()}-${device_id}`,
114     priority: result.priority,
115     reason: result.reason,
116     processed_at: new Date().toISOString(),
117   });
118 }
119
120 const grpcServer = new grpc.Server();
121 grpcServer.addService(proto.PriorityService.service, { ClassifyAlert });
122 grpcServer.bindAsync(
123   `0.0.0.0:${GRPC_PORT}`,

```

```

index.js X
c5_campus-GRPC > services > priority > index.js > classifyAlert
124   grpc.ServerCredentials.createInsecure(),
125   (err, port) => {
126     if (err) { console.error(`[Priority] Error gRPC:`, err); process.exit(1); }
127     console.log(`[Priority] ✓ gRPC servidor escuchando en puerto ${port}`);
128     console.log(`[Priority] ✓ Esperando llamadas de Geolocation...`);
129   }
130 );
131
132 app.get("/health", (req, res) => {
133   res.json({
134     status:      "ok",
135     service:     "priority",
136     grpc_port:   GRPC_PORT,
137     rules_version: rulesVersion,
138     types_indexed: Object.keys(lookupTable).length,
139     technique:    "Hash Table O(1)",
140   });
141 });
142
143 app.get("/rules", (req, res) => {
144   res.json({
145     version:      rulesVersion,
146     lookup_table: lookupTable,
147     total_types:  Object.keys(lookupTable).length,
148   });
149 });
150
151 app.get("/classify", (req, res) => {
152   const { type } = req.query;
153   if (!type) return res.status(400).json({ error: "Se requiere ?type=<tipo>" });
154
155   const result = classifyAlert(type);
156   res.json({
157     input:        type,
158     ...result,
159     classification_us: (result.classification_ns / 1000).toFixed(3) + " microsegundos",
160   });
161 });
162
163 app.put("/rules", (req, res) => {
164   try {
165     const newRules = req.body;
166     fs.writeFileSync(RULES_PATH, JSON.stringify(newRules, null, 2));
167     loadRules();

```

```

JS index.js X
c5_campus-GRPC > services > priority > JS index.js > classifyAlert
163   app.put("/rules", (req, res) => {
168     res.json({ message: "Reglas actualizadas", version: rulesVersion });
169   } catch (err) {
170     res.status(500).json({ error: err.message });
171   }
172 });
173
174 loadRules();
175
176 app.listen(HTTP_PORT, () => {
177   console.log(`[Priority] HTTP API en puerto ${HTTP_PORT}`);
178   console.log(`[Priority] Técnica: Tabla Hash O(1) – clasificación en ~0.01ms`);
179 });

```

El archivo **index.js** es el componente principal del microservicio **Priority**, ya que contiene toda la lógica necesaria para clasificar las alertas de emergencia dentro del sistema C5 Campus, su función principal es recibir información sobre una emergencia, determinar su nivel de prioridad y devolver una respuesta que pueda ser utilizada por otros servicios del sistema para tomar decisiones rápidas.

Este archivo integra dos mecanismos de comunicación, el primero es un **servidor gRPC**, que permite que otros microservicios intercambien información de manera rápida y eficiente y el segundo es una **API REST**, utilizada para tareas de administración, monitoreo y pruebas del servicio.

Al iniciar la aplicación, el sistema carga las reglas definidas en el archivo rules.json, estas reglas contienen los distintos tipos de emergencias y la prioridad asociada a cada una. Posteriormente, la información es transformada en una tabla hash para permitir búsquedas extremadamente rápidas

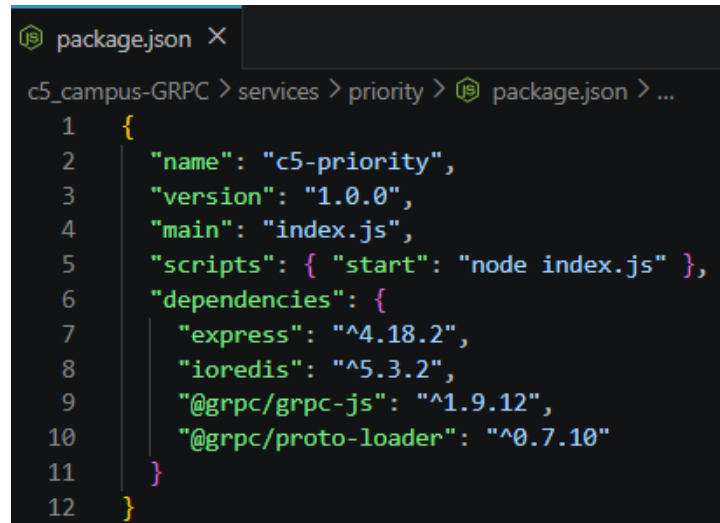
La función principal del servicio consiste en analizar el tipo de emergencia recibido y compararlo con las reglas almacenadas en memoria. Si encuentra una coincidencia, devuelve la prioridad correspondiente junto con la razón de la clasificación, en caso de que el tipo de emergencia no exista dentro de las reglas configuradas, el sistema asigna una prioridad predeterminada para garantizar que la alerta siga siendo atendida

Además de la clasificación, el archivo implementa un mecanismo de **recarga en caliente (Hot Reload)** que permite actualizar las reglas sin necesidad de reiniciar el servicio. Cuando el archivo rules.json es modificado, los cambios son detectados automáticamente y cargados nuevamente en memoria, asegurando que las nuevas configuraciones entren en funcionamiento de inmediato.

Dentro de este mismo archivo también se configura el servidor gRPC encargado de atender solicitudes provenientes de otros microservicios, especialmente del servicio de geolocalización. Cuando una alerta es recibida, el servidor procesa la información, ejecuta la clasificación y devuelve una respuesta que incluye un identificador único de alerta, el nivel de prioridad asignado, la razón de la clasificación y la fecha de procesamiento.

Por otra parte, la API REST proporciona diferentes endpoints para supervisar el estado del servicio, consultar las reglas activas, realizar pruebas de clasificación y actualizar la configuración del sistema. Estas funcionalidades facilitan las tareas de mantenimiento y administración sin afectar el funcionamiento normal del servicio.

Package.json



```
1  {
2    "name": "c5-priority",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": { "start": "node index.js" },
6    "dependencies": {
7      "express": "^4.18.2",
8      "ioredis": "^5.3.2",
9      "@grpc/grpc-js": "^1.9.12",
10     "@grpc/proto-loader": "^0.7.10"
11   }
12 }
```

El archivo **package.json** es el archivo de configuración principal del microservicio **Priority** y cumple la función de definir la información básica del proyecto, las dependencias necesarias para su funcionamiento y los comandos que permiten ejecutarlo

Dentro de este archivo se establece el nombre del proyecto mediante la propiedad **name**, identificándolo como **c5-priority**, lo que facilita su reconocimiento dentro de la arquitectura de microservicios del sistema C5 Campus.

Otra configuración importante es la propiedad **main**, la cual especifica que el archivo principal de ejecución es **index.js**, esto significa que cuando la aplicación se inicia, Node.js comenzará la ejecución desde dicho archivo, donde se encuentra toda la lógica relacionada con la clasificación de alertas, la configuración del servidor gRPC y la API REST.

El archivo también incluye una sección denominada **scripts**, que contiene comandos personalizados para facilitar la ejecución del proyecto, en este caso se define el comando **start**, que ejecuta automáticamente el archivo principal mediante la instrucción `node index.js`.

Además, el archivo **package.json** administra las dependencias externas que requiere el sistema para funcionar correctamente, entre ellas se encuentra **Express**, una biblioteca utilizada para crear la API REST que permite consultar el estado del servicio, administrar las reglas y realizar pruebas de clasificación, también se incluye **ioredis**, una biblioteca que permite establecer conexiones con servidores Redis y que deja preparada la aplicación para futuras integraciones relacionadas con almacenamiento temporal o mensajería.

Por otra parte, el proyecto utiliza **@grpc/grpc-js**, la implementación oficial de gRPC para Node.js, la cual permite establecer la comunicación eficiente entre microservicios mediante llamadas remotas, complementando esta funcionalidad, se emplea la biblioteca **@grpc/proto-loader**, encargada de cargar e interpretar el archivo `priority.proto`, donde se definen los mensajes y servicios utilizados en la comunicación gRPC.

En conjunto, el archivo **package.json** funciona como el centro de configuración del microservicio Priority, ya que permite identificar el proyecto, controlar su versión, definir cómo debe ejecutarse y gestionar todas las bibliotecas necesarias para que el sistema pueda comunicarse mediante gRPC, ofrecer servicios REST y realizar la clasificación de alertas de manera eficiente.

Priority.proto

```
priority.proto X
c5_campus-GRPC > services > priority > priority.proto
1  syntax = "proto3";
2
3  package priority;
4
5  service PriorityService {
6    rpc ClassifyAlert (AlertRequest) returns (AlertResponse);
7  }
8
9  message AlertRequest {
10     string device_id      = 1;
11     string emergency_type = 2;
12     string zone           = 3;
13     double latitude       = 4;
14     double longitude      = 5;
15     string timestamp      = 6;
16  }
17
18  message AlertResponse {
19     string alert_id       = 1;
20     string priority       = 2;
21     string reason         = 3;
22     string processed_at   = 4;
23  }
```


El archivo `priority.proto` es un componente fundamental dentro del microservicio `Priority`, ya que define el contrato de comunicación utilizado por gRPC entre este servicio y los demás microservicios del sistema.

Su función principal es establecer de manera formal qué información puede enviarse, qué información será recibida como respuesta y qué métodos estarán disponibles para la comunicación remota.

Gracias a este archivo, tanto el cliente como el servidor conocen exactamente la estructura de los datos que intercambiarán, evitando errores de interpretación y garantizando una comunicación eficiente.

La primera línea del archivo utiliza la instrucción `syntax = "proto3";`, la cual indica que se empleará la versión 3 de Protocol Buffers. Protocol Buffers es el mecanismo de serialización de datos desarrollado por Google y utilizado por gRPC para transmitir información de forma compacta y rápida.

Esta tecnología permite que los mensajes ocupen menos espacio que formatos tradicionales como JSON o XML, mejorando el rendimiento de la comunicación entre servicios.

Posteriormente se define el paquete mediante la instrucción `package priority;`. Este paquete funciona como un espacio de nombres que agrupa todos los elementos relacionados con el servicio de prioridad, su objetivo es evitar conflictos con otros servicios que puedan tener mensajes o métodos con nombres similares dentro de la arquitectura del sistema.

Dentro del archivo se encuentra la definición del servicio llamada `PriorityService`. Un servicio en gRPC representa el conjunto de operaciones que un cliente puede invocar remotamente.

En este caso, el servicio expone un único método denominado `ClassifyAlert`, cuya responsabilidad es recibir la información de una alerta de emergencia y devolver la clasificación correspondiente. Este método actúa como el punto de entrada principal para que otros microservicios soliciten la determinación de la prioridad de una emergencia.

Para enviar la información al servicio se utiliza el mensaje `AlertRequest`, el cual contiene todos los datos necesarios para procesar una alerta.

Entre estos datos se encuentra el identificador del dispositivo o tótem que generó la alerta, almacenado en el campo `device_id`.

También incluye el campo `emergency_type`, que representa el tipo de emergencia reportada, como por ejemplo incendio, robo o emergencia médica. Adicionalmente se

recibe la zona donde ocurrió el evento mediante el campo `zone`, así como las coordenadas geográficas representadas por la latitud y longitud

Finalmente, el mensaje incorpora una marca de tiempo mediante el campo `timestamp`, que indica cuándo fue generada la alerta.

Una vez procesada la información, el servicio devuelve un mensaje de tipo `AlertResponse`, este mensaje contiene un identificador único de alerta denominado `alert_id`, el cual permite rastrear el evento dentro del sistema. También incluye el campo `priority`, donde se almacena el nivel de prioridad asignado por el servicio, pudiendo ser crítica, alta o media según las reglas configuradas. Además, se proporciona una explicación de la clasificación mediante el campo `reason`, que describe por qué la alerta recibió determinada prioridad

Finalmente, el campo `processed_at` registra la fecha y hora exactas en que la alerta fue procesada por el sistema.

En términos generales, el archivo `priority.proto` funciona como la especificación oficial de la comunicación entre servicios dentro del sistema C5 Campus

Gracias a él, el microservicio `Priority` puede intercambiar información de forma rápida, estructurada y segura con otros componentes de la arquitectura, garantizando que todas las alertas de emergencia sean procesadas bajo un formato estándar y consistente.

Rules.json

```
{ } rules.json X
c5_campus-GRPC > services > priority > { } rules.json > ...
1  {
2    "_comment": "Reglas de prioridad para Campus Universitario Inteligente. Recarga en caliente.",
3    "_version": "2.0-campus",
4    "_context": "Sistema de seguridad universitaria – tótems distribuidos en campus",
5
6    "critical": {
7      "label": "CRÍTICO",
8      "response_time_seconds": 120,
9      "color": "#EF4444",
10     "types": [
11       "robo_con_violencia",
12       "secuestro",
13       "tiroteo",
14       "explosion",
15       "ataque_terrorista",
16       "rehenes",
17       "amenaza_bomba"
18     ],
19     "reason": "Riesgo inmediato para la comunidad universitaria – respuesta en menos de 2 minutos"
20   },
21
22   "high": {
23     "label": "ALTO",
24     "response_time_seconds": 300,
25     "color": "#F97316",
26     "types": [
27       "incendio",
28       "accidente_grave",
29       "acoso_violencia",
30       "emergencia_medica",
31       "fuga_gas",
32       "pelea_grupal",
33       "persona_herida",
34       "agresion_sexual"
35     ],
36     "reason": "Emergencia que requiere respuesta inmediata – máximo 5 minutos"
37   },
38 }
```

```
{ } rules.json X
c5_campus-GRPC > services > priority > { } rules.json > ...
39   "medium": {
40     "label": "MEDIO",
41     "response_time_seconds": 900,
42     "color": "#EAB308",
43     "types": [
44       "robo",
45       "vandalismo",
46       "sospechoso",
47       "acoso_verbal",
48       "persona_extraviada",
49       "accidente_menor",
50       "falla_electrica",
51       "persona_vulnerable"
52     ],
53     "reason": "Situación que requiere atención – máximo 15 minutos"
54   },
55
56   "default_priority": "high",
57   "default_reason": "Emergencia en campus sin clasificación específica – prioridad alta por precaución"
58 }
59
```

El archivo **rules.json** es el encargado de almacenar todas las reglas de negocio utilizadas por el microservicio Priority para clasificar las alertas de emergencia, su principal función es definir qué tipos de incidentes existen dentro del sistema y qué nivel de prioridad debe asignarse a cada uno.

Gracias a este archivo, la lógica de clasificación puede modificarse sin necesidad de cambiar el código fuente de la aplicación, permitiendo que las reglas sean actualizadas de forma rápida y flexible.

Además de contener las categorías de prioridad, el archivo incluye información descriptiva sobre el sistema, como comentarios de documentación, la versión actual de las reglas y el contexto de uso dentro del Campus Universitario Inteligente

Estos datos sirven para identificar fácilmente la configuración activa y facilitar su mantenimiento a lo largo del tiempo.

Las reglas están organizadas en tres niveles principales de prioridad: **critical**, **high** y **medium**, cada uno de estos niveles contiene un conjunto de atributos que ayudan al sistema a determinar cómo debe gestionarse una alerta, entre estos atributos se encuentra una etiqueta descriptiva que se utiliza para mostrar el nivel de prioridad de manera legible, un tiempo máximo de respuesta expresado en segundos, un color asociado para facilitar la visualización en interfaces gráficas y una lista de tipos de emergencias que pertenecen a dicha categoría.

La categoría **critical** representa las situaciones de mayor riesgo para la comunidad universitaria, dentro de esta clasificación se encuentran eventos como secuestros, tiroteos, explosiones, amenazas de bomba o situaciones con rehenes

Estas emergencias requieren una respuesta inmediata debido al peligro que representan para la vida de las personas, por lo que el sistema establece un tiempo de respuesta máximo de dos minutos, cuando una alerta coincide con alguno de estos tipos, el servicio asigna automáticamente la prioridad crítica.

La categoría **high** agrupa incidentes graves que, aunque no representan una amenaza extrema como las anteriores, requieren atención urgente. En este grupo se incluyen incendios, accidentes graves, emergencias médicas, fugas de gas, agresiones sexuales o peleas grupales. Para estos casos el sistema establece un tiempo máximo de respuesta de cinco minutos, garantizando una intervención rápida por parte de las autoridades o personal de seguridad.

Por su parte, la categoría **medium** contiene situaciones que siguen siendo importantes, pero cuyo nivel de urgencia es menor en comparación con las categorías anteriores. Ejemplos de estas alertas son robos sin violencia, actos de vandalismo, personas sospechosas, accidentes menores o fallas eléctricas. Para estos eventos se establece

un tiempo máximo de respuesta de quince minutos, permitiendo una atención adecuada sin comprometer recursos destinados a emergencias más críticas.

El archivo también define una configuración de respaldo mediante los campos **default_priority** y **default_reason**, estos valores se utilizan cuando el sistema recibe un tipo de emergencia que no se encuentra registrado dentro de las reglas existentes. En lugar de rechazar la alerta, el servicio asigna automáticamente una prioridad predeterminada y una explicación asociada.

En este caso, la prioridad por defecto es **high**, aplicando el principio de precaución para evitar que una posible emergencia importante quede sin atención.

Una característica importante de este archivo es que puede ser modificado sin necesidad de reiniciar el microservicio, el sistema monitorea constantemente los cambios realizados en `rules.json` y recarga automáticamente la información cuando detecta una actualización.

Esto permite agregar nuevos tipos de emergencias, modificar prioridades o ajustar tiempos de respuesta de manera inmediata, garantizando que el sistema pueda adaptarse rápidamente a nuevas necesidades operativas.

En conclusión, el archivo **rules.json** constituye la base de conocimiento del microservicio Priority, ya que concentra todas las reglas utilizadas para clasificar las alertas de emergencia.

Su diseño permite mantener separada la lógica de negocio del código fuente, facilitando la administración, actualización y escalabilidad del sistema de seguridad del Campus Universitario Inteligente.